

Connect Four 2.0

Real-Time Multiplayer Edition

Technical Documentation

Backend: Java, Spring Boot, STOMP over WebSocket
Frontend: React (AI-assisted)
Author: NVP Manohar Kancharla

1. Overview

Connect Four 2.0 is a real-time, two-player version of the classic game. Players create or join a room using a short room code, take turns dropping discs into a 7-column, 6-row board, and chat with each other while playing. The server is the single source of truth: it validates every move, decides turn order and detects wins or draws, then broadcasts the updated state to both players.

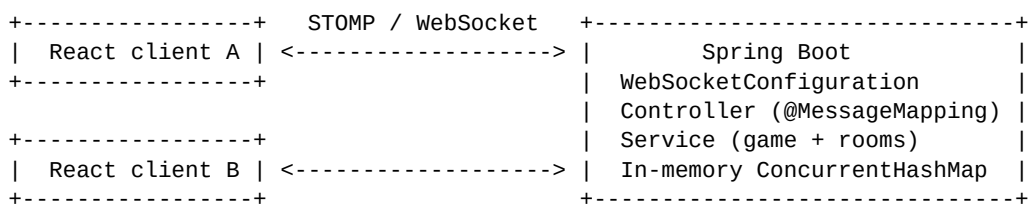
Version 2.0 builds on the earlier single-player release, which used a REST API. The move to WebSocket messaging allows both clients to be updated instantly without polling.

Key features

- Room-based matchmaking with shareable room codes
- Live board synchronisation between two players over STOMP/WebSocket
- Server-side validation of moves, turns, wins (horizontal, vertical, both diagonals) and draws
- In-room chat alongside the board (70% board / 30% chat layout)
- Error propagation to the offending client (invalid move, wrong turn, full column, unknown room)
- Animated UI: disc drop with bounce, pulsing highlight on winning cells

2. Architecture

The system is a standard client-server design. The React client keeps a single WebSocket connection to the Spring Boot server and communicates through STOMP frames. All game state lives in memory on the server.



Layers

Layer	Responsibility
WebSocketConfiguration	Registers the STOMP endpoint, enables the simple message broker, and sets application/broker destination prefixes.
Controller	Receives client messages via @MessageMapping, extracts the room code with @DestinationVariable, delegates to the service, and publishes results to the room topic.
Service	Owns the rooms and game rules. Creates rooms, seats players, applies moves, detects win/draw.
Exception handling	@MessageExceptionHandler inside a @ControllerAdvice converts exceptions into error messages delivered to the sender.
Model / DTOs	Room, Game, Board, Player, Move and chat message objects exchanged as JSON.

3. Technology Stack

Area	Technology
Language	Java
Backend framework	Spring Boot on Spring Framework 7
Messaging	STOMP over WebSocket (Spring messaging)
State	In-memory: ConcurrentHashMap with synchronized service methods
Frontend	React
Deployment	Render

4. Game Flow

- **Create room:** Player 1 requests a room. The server generates a room code and seats the player as the first colour (red).
- **Join room:** Player 2 enters the code. The server seats them as the second colour (yellow) and the game becomes active.
- **Play:** The client sends a move (column index). The server checks that the room exists, the game is active, it is the sender's turn and the column is not full. It then drops the disc into the lowest free row.
- **Broadcast:** The updated board, current turn and status are published to every subscriber of the room topic.
- **Finish:** On four in a row the status becomes a win and the winning cells are included in the state so the UI can highlight them. A full board with no winner is a draw.
- **Chat:** Chat messages are published to the same room, so only the two players see them.

5. Messaging Contract

The server exposes three kinds of interaction. Exact destination names are defined in the controller and the frontend WebSocket service; they follow the pattern below.

Interaction	Direction	Payload / Result
Create / join room	Client to server	Player name and (for join) the room code. Server replies with the room state.
Make move	Client to server (room-scoped destination)	Column index. Room code is taken from the destination path via <code>@DestinationVariable</code> .
Game state update	Server to all room subscribers	Board grid, current turn, status (waiting / in progress / won / draw), winner, winning cells.
Chat	Client to server, then broadcast	Sender name and message text.
Error	Server to the offending client	Human-readable error message (invalid move, not your turn, room full, etc.).

6. Backend Design Notes

Concurrency

Two players can act on the same room at nearly the same time. Rooms are stored in a `ConcurrentHashMap` so lookups and insertions are thread-safe, and the service methods that mutate a game (join, move) are synchronized. This guarantees turn order cannot be violated by racing requests.

Spring Framework 7 specifics

- In STOMP handlers, path variables must be bound with `@DestinationVariable`, not `@PathVariable` (the latter is for HTTP and fails in message handlers).
- A circular bean dependency in `WebSocketConfiguration` was resolved by removing the configuration's dependency on components that themselves depend on the messaging template.
- STOMP error propagation uses `@ExceptionHandler` placed in a `@ControllerAdvice`, so errors reach only the client that caused them.

Known limitations

- State is in memory: restarting the server (or a Render free-tier spin-down) drops all active rooms.
- Single-instance only. Scaling out would require an external broker (e.g. Redis or RabbitMQ) and shared state.
- No authentication; players are identified by the name they enter for the session.
- No reconnection handling: a dropped client must rejoin with the room code.

7. Frontend

The React client is a single-page app. **Note:** the frontend was built with AI assistance; the backend and game logic are the author's own work.

- **Layout:** board occupies about 70% of the width, chat panel about 30%.
- **Theme:** navy gradient background, red and yellow discs.
- **Animation:** disc drop with bounce; pulsing winning cells.
- **Room UI:** the room code is displayed prominently so it can be shared.
- **State:** the client renders whatever the latest server state is; it does not compute game rules itself.

8. Setup and Running Locally

Prerequisites

- JDK (version required by your Spring Boot 4 / Spring Framework 7 build)
- Maven or Gradle wrapper (included in the project)
- Node.js and npm

Backend

```
cd backend
./mvnw spring-boot:run
```

Frontend

```
cd frontend
npm install
npm run dev
```

Open the frontend URL in two browser windows (or two devices), create a room in one and join with the code in the other.

9. Deployment (Render)

- Deploy the backend as a Web Service and the frontend as a Static Site (or a second Web Service).
- Point the frontend's WebSocket URL at the deployed backend, using `wss://` on HTTPS.
- Allow the frontend's origin in the backend's WebSocket allowed-origins setting.
- Set the Node.js version explicitly on Render (for example via the `NODE_VERSION` environment variable or an `engines` entry in `package.json`) to avoid build failures from a version mismatch.
- On free tiers the service sleeps when idle; the first connection after a pause can take a while.

10. Testing Checklist

- Create a room and confirm a code is shown
- Join with a valid code, then with an invalid code (error expected)
- Third player attempting to join a full room is rejected
- Moving out of turn is rejected only for the sender
- Dropping into a full column is rejected
- Win detection: horizontal, vertical, both diagonals
- Draw when the board is full
- Chat appears for both players and no one else
- Refresh mid-game behaves as documented (room is kept server-side, client must rejoin)

11. Future Improvements

- Reconnection and session tokens
- Persistent storage (database) and match history
- External message broker for horizontal scaling
- Rematch button and spectator mode
- Player accounts and a leaderboard
- Automated tests for the win-detection logic and STOMP flows

12. Credits

Backend architecture, game logic and debugging: NVP Manohar Kancharla. Frontend implementation: AI-assisted (Claude by Anthropic), reviewed and integrated by the author.